barrier/dissemination

An AgentMPI collective scaling analysis

Abstract

The dissemination barrier synchronises p ranks without a root: in round k every rank sends a one-token token to rank $(r + 2^k) \bmod p$ and receives from $(r - 2^k) \bmod p$, and after $\lceil \log_2 p \rceil$ rounds every rank has transitively heard from every other. The sweep confirms the message count $p\lceil \log_2 p \rceil$ at all twenty-one measured sizes — 24 at $p = 8$, 64 at $p = 16$, 160 at $p = 32$ — and confirms the symmetry: at every size, every rank sent the same number of messages. It also exposes a defect. **The `rounds` field of every `coll.barrier` record in the repository is zero.** `barrier()` computes its round count, returns it in `BarrierResult`, and never assigns it to `tr.stats.rounds`; it is the only collective in `algorithms.py` that omits that assignment, and all forty-two barrier sweep runs plus all four barriers in the real eight-rank software run report 0 rounds against predictions of 3 and 5. For the one collective whose entire cost is latency and whose payload is a single token, the round count is the only number that matters, and it is the one number the trace does not carry. Beyond that, the important point about barriers for agent ranks is not the algorithm at all: a barrier costs what its straggler costs, and in `real-sw-p8-full` one integration barrier held ranks for between 0 and 215.6 s each, 924.8 rank-seconds in total. These agent-free runs cannot show that, and any reading of them as a statement about agent synchronisation would be wrong.

1 What this family is

The `barrier` collective under the `dissemination` algorithm, measured across 21 runs at 13 distinct process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.013–0.236 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

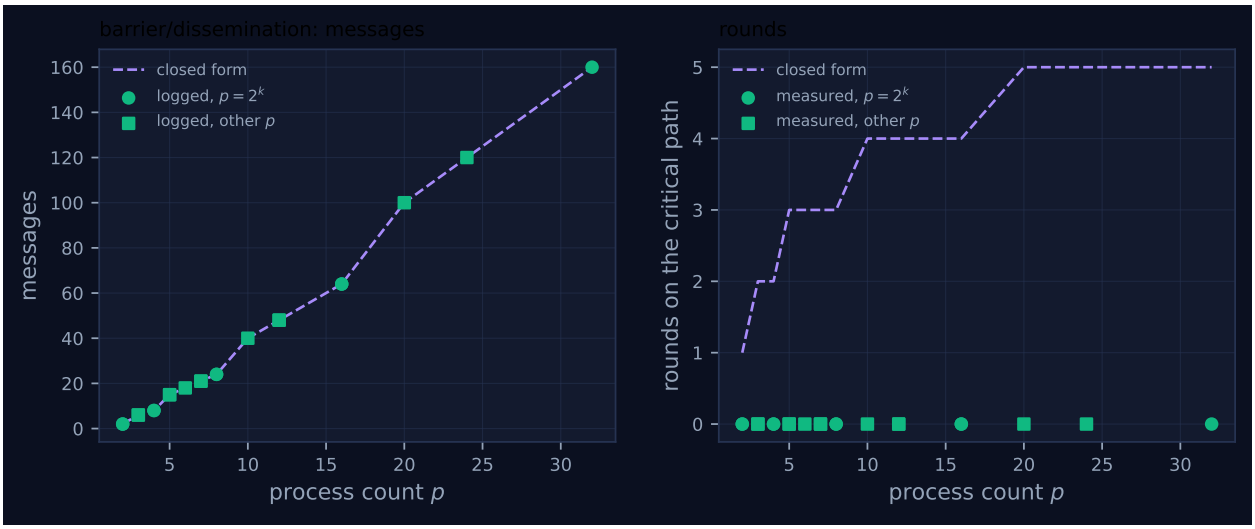

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	0	1	0.014	✓	✓
2	`mb-smoke`	2	2	2	0	1	0.013	✓	✓
3	`mb-free`	6	6	6	0	2	0.018	✓	✓
3	`mb-smoke`	6	6	6	0	2	0.017	✓	✓
4	`mb-free`	8	8	8	0	2	0.025	✓	✓
4	`mb-smoke`	8	8	8	0	2	0.017	✓	✓
5	`mb-free`	15	15	15	0	3	0.035	✓	✓
5	`mb-smoke`	15	15	15	0	3	0.027	✓	✓
6	`mb-free`	18	18	18	0	3	0.039	✓	✓
7	`mb-free`	21	21	21	0	3	0.039	✓	✓
7	`mb-smoke`	21	21	21	0	3	0.036	✓	✓
8	`mb-free`	24	24	24	0	3	0.045	✓	✓
8	`mb-smoke`	24	24	24	0	3	0.039	✓	✓
10	`mb-free`	40	40	40	0	4	0.051	✓	✓
12	`mb-free`	48	48	48	0	4	0.073	✓	✓
12	`mb-smoke`	48	48	48	0	4	0.081	✓	✓
16	`mb-free`	64	64	64	0	4	0.171	✓	✓
16	`mb-smoke`	64	64	64	0	4	0.105	✓	✓
20	`mb-free`	100	100	100	0	5	0.120	✓	✓
24	`mb-free`	120	120	120	0	5	0.162	✓	✓
32	`mb-free`	160	160	160	0	5	0.236	✓	✓

4 How the algorithm scales

The algorithm is eleven lines and its cost is legible in them. For k in $0 \ldots \lceil \log_2 p \rceil - 1$, rank r sends one token to $(r + 2^k) \bmod p$ and receives one from $(r - 2^k) \bmod p$, both under a tag that includes k so that rounds cannot be confused with one another. After round k each rank has heard,

transitively, from the 2^k ranks preceding it; after $\lceil \log_2 p \rceil$ rounds the reach covers all p. Every rank does the same work, so the message count is $p\lceil \log_2 p \rceil$ and the critical path is $\lceil \log_2 p \rceil$ rounds — exactly `FORMULAS["barrier","dissemination"]`, whose volume term is also $p\lceil \log_2 p \rceil$ because the payload is the integer 1.

The message panel of Figure 1 is that expression and matches at every one of the twenty-one rows: 6 at $p = 3$, 15 at $p = 5$, 24 at $p = 8$, 40 at $p = 10$, 64 at $p = 16$, 100 at $p = 20$, 120 at $p = 24$, 160 at $p = 32$. It rises in a staircase because $\lceil \log_2 p \rceil$ is constant between powers of two — 24, 40, 48 and 64 for $p = 8, 10, 12, 16$ — multiplied by a p that rises smoothly, so the curve is piecewise linear with a slope change at each power of two. Where the round count steps is therefore visible in the message panel as a kink, and the kinks land at 2, 4, 8, 16 and 32.

The round panel is where the family view earns its keep, and what it shows is a defect. The measured round count is 0 at all twenty-one sizes, against a closed form that rises from 1 to 5. That is not a scaling result about the algorithm; it is missing instrumentation. In `barrier()` the local variable `rounds` is set to `n_rounds` for dissemination, to 2 for `linear` and to 2 for `central`, and it is returned to the caller inside `BarrierResult` — but `tr.stats.rounds` is never assigned, so the `coll.barrier` event that `_Tracker.finish` emits carries the dataclass default of 0. Every other algorithm in the module assigns it; there are twenty-five such assignments across `bcast`, `scatter`, `gather`, `allgather`, `reduce`, `allreduce`, `scan`, `reduce_scatter` and `alltoall`, and none in `barrier`. I checked every barrier trace in the repository: all forty-two runs of the two sweep families report `"rounds": 0`, and so do all four barriers in `real-sw-p8-full`, where `metrics.json` accordingly records `"rounds_agree": false` against `predicted_rounds` of 3 for the two dissemination fences and 2 for the two central barriers.

Why it survived is instructive. The benchmark that produced these runs, `bench_collectives` in `experiments/microbench.py`, does read `rounds_measured` and `rounds_model` into its result rows — and then defines its mismatch list as the rows failing `messages_match` or `fold_depth_match`. Rounds are recorded and never compared. The family table here is the first artefact in the repository that puts the measured and predicted round counts in adjacent columns, and the disagreement is unmissable the moment it does. The severity is worth being precise about: the messages were sent correctly, the barrier synchronised correctly, and the only thing wrong is the report. But for a barrier that is the whole measurement. Its payload is one token, its message count is arithmetic, and the quantity a harness author needs when choosing between $\lceil \log_2 p \rceil$ and $2(p-1)$ is exactly the number that is zero.

What the trace does carry is the per-rank symmetry, and it is worth stating because it is the algorithm's distinguishing property. At every size, every rank's `coll.barrier` record reports the same `messages_sent` — 3 each at $p = 8$, 5 each at $p = 32$ — and the viewer screenshot for `mb-free__coll-barrier-dissemination-32` shows thirty-two lanes of indistinguishable density, roughly ten glyphs each, with no lane carrying a bar and no lane carrying more traffic than another. Set that beside the screenshot for `mb-free__coll-barrier-linear-32`, where rank 0's lane is the only one with thick bars, and the difference between "no root" and "a root" is a picture rather than a number. That is what the viewer supplies that the metrics lost.

5 Powers of two and everything else

Dissemination is correct for every p and not only for powers of two — the docstring says so, and the reason is that modular arithmetic needs no padding: $(r \pm 2^k) \bmod p$ is defined for any p, and

after $\lceil \log_2 p \rceil$ rounds the reach $2^{\lceil \log_2 p \rceil} \geq p$ covers the whole group whether or not it divides evenly. That is why MPI implementations favour it, and it is why this family has no remainder path to go wrong in.

The measurements confirm it without qualification. The squares in Figure 1 lie on $p\lceil \log_2 p \rceil$ as exactly as the circles: 6, 15, 18, 21, 40, 48, 100 and 120 at $p = 3, 5, 6, 7, 10, 12, 20, 24$, each matching the closed form to the message. There are no red crosses, so the self-reported count equalled the logged traffic at all twenty-one sizes, and the eight sizes measured twice — $p = 2, 3, 4, 5, 7, 8, 12, 16$ in both `mb-free` and `mb-smoke` — agree on every count, differing only in wall time (0.045 s against 0.039 s at $p = 8$). Two independent executions agreeing at the same p is the cheapest evidence available that these are properties of the algorithm rather than of one run's scheduling.

One consequence of the no-padding property deserves emphasis because it distinguishes this algorithm from most of the sweep. At a non-power-of-two size the reach overshoots: at $p = 5$, three rounds give a reach of 8 against a need of 5, so some ranks hear from some peers twice. That redundancy is not wasted work the implementation could avoid — it is what makes the algorithm size-agnostic — and it is why the message count is $p\lceil \log_2 p \rceil$ rather than something smaller like the $p\lceil \log_2 p \rceil - (2^{\lceil \log_2 p \rceil} - 1)$ that `scan/recursive_doubling` gets to claim by skipping sends with no destination. A barrier has no such option, because every send has a destination; the overshoot is in duplicated information, not in wasted messages.

6 When to choose this algorithm

Against `linear`, on paper, dissemination wins decisively at scale and the sweep gives both sides of it. At $p = 32$: 5 rounds against $2(p - 1) = 62$ on the critical path, 160 messages against 62, and a perfectly flat per-rank load (5 messages each) against a root that handles 62 of the 62. Under the agent cost model those numbers are not close. `cost.predict("barrier", ..., p=32)` with 4,096-token context gives 2.58 s for dissemination against 31.93 s for linear, a factor of 12.4, and the ratio is $2(p - 1)/\lceil \log_2 p \rceil$, so it widens with p.

Measured on this fabric the ranking inverts, and saying so is more useful than hiding it. At $p = 32$ dissemination spans 0.236 s and accumulates 4.30 rank-seconds of receive-blocking, while linear spans 0.102 s and accumulates 0.82. The reason is that with no agents in the loop the per-round cost is a few hundred microseconds of SQLite rather than seconds of invocation latency, so the term that dominates is the message count, and dissemination sends 2.6 times more messages than linear at $p = 32$. In other words this sweep measures the regime where $\alpha \to 0$, in which a latency-optimal algorithm has nothing to optimise and pays for its redundancy. The crossover is where per-round cost exceeds roughly $(160 - 62)/(62 - 5) \approx 1.7$ times the per-message cost, which for any real agent invocation — α on the order of 10^0–10^1 s against a fabric round trip of 0.003 s — is met by three orders of magnitude. Take dissemination for agent ranks; the sweep does not contradict that, it simply measures the other end of the axis.

There is a third algorithm in `FORMULAS` that the sweep never exercises and that a harness should usually prefer, and the reason is policy rather than cost. `central` registers arrival in the fabric and polls the count: two round trips, zero messages, and — uniquely — it can *name* the ranks that failed to arrive, which no message-passing barrier can. `barrier()` therefore coerces `algorithm = "central"` whenever the policy is `PROCEED`, `SHRINK` or `REVOKE`, so this family only exists because `bench_collectives` passes `policy=BarrierPolicy.WAIT` explicitly. That is the sweep's most important configuration choice and it should be read as a constraint on generalisation: the four

barrier policies AgentMPI adds to MPI are exercised by the `mb-free__faults-*` runs, not here. In those runs, with rank 3 dead, `PROCEED` and `SHRINK` both completed after 15.2 s having sent zero messages and correctly reported `"absent": [3]`, while `WAIT` and `RAISE` produced no `coll.barrier` record at all — 17 sends against 10 receives, seven messages left undelivered, the last send at 0.03 s and the seven surviving ranks finalising together at 15.25 s. Those runs exercise this algorithm: the tags run `barrier:0:1:0` through `:2`, three rounds for $p = 8$, and `bench_faults` names no algorithm so it takes dissemination by default. A barrier that blocks forever is how agent harnesses hang, and `mb-free__faults-wait` is what that looks like in a trace: a dissemination pattern truncated mid-round and no completion record to say a barrier was ever attempted.

7 Threats to this reading

These runs contain no agents, so they say nothing about synchronisation cost. The executor is unset, the viewer reports zero agent calls, and the 0.013–0.236 s wall times are SQLite writes and event-log appends. This matters more for `barrier` than for any other collective in the sweep, because a barrier's cost is by definition the cost of its slowest participant, and agent latency is heavy-tailed in a way that thread scheduling is not. The repository's own cost model records this distinction explicitly: `CostParams` carries both `alpha_p50` and `alpha_p99` on the grounds that "the *spread* of α — not its mean — governs how long a barrier waits". Nothing in this family measures that spread.

Where it is measured is the real-agent runs, and the contrast is the whole point. In `real-sw-p8-full` the `integrate-r1` barrier held its eight ranks for 0.00, 22.45, 60.72, 117.25, 140.02, 177.54, 191.30 and 215.56 seconds respectively — 924.8 rank-seconds in total, a 9.6-fold spread between the fastest and slowest waiter, and the last arrival setting the release time for everyone. The dissemination fence labelled `win_fence:interfaces-published` in the same run shows 0.00 to 43.49 s, a 6.0-fold spread over 162.3 rank-seconds. Across that run's four barriers, 1,574 of the 1,666 rank-seconds spent in *all* collectives were spent in barriers — 94 % — which is 39 % of the entire run's rank-time. This family's twenty-one runs contain nothing comparable, and a reader should take the message and round counts from here and the latency story from there.

The apparent arrival spread in the sweep is not an agent result. The `coll.barrier` records do report per-rank wall times, and at $p = 32$ they range from 0.0012 s to 0.2116 s, a factor of 176. It is tempting to read that as heavy-tailed arrival, and it would be wrong: the quantity is the interval each rank spent inside the barrier, so the rank that arrives last reports nearly zero and the rank that arrived first reports the whole span. The 176-fold ratio therefore measures thread start-up jitter and SQLite contention across thirty-two Python threads, and nothing about model latency.

The round-count defect is an omission, and I have not observed a consequence of it. The claim is that `tr.stats.rounds` is never assigned in `barrier()`; the evidence is the absence of the assignment in the source, the presence of it in all twenty-five other algorithm branches, and `"rounds": 0` in all forty-two sweep runs plus all four real-run barriers. What I cannot show is harm: no result in the repository appears to have been computed from a barrier's round count, precisely because it has always been zero. The risk is prospective — a calibration or algorithm-selection step that reads `rounds` would conclude a barrier is free — and the fix is one line.

Twenty-one rows, thirteen distinct sizes, one or two runs each. Nothing here is averaged over repetitions, so wall times should be read as single samples; the integer counts, being deterministic, do not need repetition, and the duplicated sizes confirm that.